

Prolog built-ins and their descriptions

Total number of documented built-ins: 817

1. **!/0** : *succeeds like true/0, but removes pending choices in calls at its LEFT and makes things look as if this were the LAST clause of the predicate*
2. **#/3** : *bitwise XOR*
3. **EXAMPLE (S)** :
4. `?-#(1,2,_A).`
5. `_A = 3;`
- 6.
7. `no`
8. **(##)/1** : *executes arg 1 at compile time - make sure the executed code terminates*
9. **(#*)/3** : *(X): adds intuitionistic assumption *(X) to be used indefinitely by #-operation*
10. **(#+)/3** : *(X): adds linear assumption +(X) to be consumed at most once, by a #-operation*
11. **(#-)/3** : *(X): consumes +(X) linear assumption or matches *(X) intuitionistic assumption*
12. **(#:)/3** : *(X): matches X against current dcg token*
13. **(#<)/3** : *(Xs): sets the dcg token list to be Xs*
14. **(#=)/3** : *(X): unifies X with any matching existing or future +(X) linear assumptions*
15. **(#>)/3** : *(Xs): unifies current dcg token list with Xs*
16. **(#?)/3** : *(X): matches +(X) or *(X) assumptions without any binding*
17. **\$catch_looking_for_throw/1** : *(CatchThrowData): continuation marker used by catch/throw*
18. **\$hx/3** : *hidden compiled hashed database predicate*
19. **\$process_catch/3** : *processes data sent by catch for throw*
20. ***/3** : *multiply*
21. **EXAMPLE (S)** :
22. `?-*(10,3,_A).`
23. `_A = 30;`
- 24.
25. `no`
26. ****/3** : *returns arg 1 at power arg 2, a float*
27. **EXAMPLE (S)** :
28. `?-**(2,3,_A).`
29. `_A = 8;`
- 30.
31. `no`
32. **(+)/2** : *returns 0 + arg 1*
33. **(+)/3** : *add*
34. **EXAMPLE (S)** :
35. `?-+(10,3,_A).`
36. `_A = 13;`
- 37.
38. `no`
39. **(+)/3** : *concatenates N lists, usable in is/2*

40. (,)/2 : *A,B succeeds if A succeeds and B, called after A, succeeds*

41. (-)/2 : *returns 0 - arg 1*

42. (-)/3 : *subtract*

43. **EXAMPLE (S) :**

44. ?--(10,3,_A) .

45. _A = 7;

46. .

47. no

48. (->)/2 : *Cond->Then executes Cond once; if it succeeds it also executes Then*

49. (,)/3 : *(Head,Tail,List) builds List=[Head\Tail]*

50. / /3 : *division*

51. **EXAMPLE (S) :**

52. ?-/ (10,3,_A) .

53. _A = 3.33333333;

54. .

55. no

56. // /3 : *integer division*

57. **EXAMPLE (S) :**

58. ?-// (10,3,_A) .

59. _A = 3;

60. .

61. no

62. ^ /3 : *bitwise AND*

63. **EXAMPLE (S) :**

64. ?-/\ (1,2,_A) .

65. _A = 0;

66. .

67. no

68. : /2 : *M:P calls predicate P hidden in module M*

69. (::)/2 : *variant of :- for hand transformed binary clauses - use with care*

70. (;)/2 : *A;B succeeds if A succeeds or B, called after A, succeeds*

71. (<)/2 : *numeric comparison*

72. << /3 : *left shifts arg 1 by arg 2 bits*

73. **EXAMPLE (S) :**

74. ?-<< (1,5,_A) .

75. _A = 32;

76. .

77. no

78. (=)/2 : *(X,Y) true if (possibly cyclic) terms X and Y unify - cyclic terms can result from =/2, as occur check is not performed*

79. **EXAMPLE (S) :**

80. ?-f(_A,s(a)) = f(_B,_B) .

81. _A = s(a);

82. _B = s(a);

83. .

84. no

85. (=..)/2 : *called univ -this is bidirectional- it converts between a term and its view as a alist of components*

86. **EXAMPLE (S) :**

87. ?-f(a,b) =.. _A .

88. _A = [f,a,b];

89. .

90. no

```

91. ?-_A =.. [f,a,b].
92. _A = f(a,b);
93.
94. no
95. (:=)/2 : numeric comparison
96. (<=)/2 : numeric comparison
97. (==)/2 : true if args are identical terms
98. (=\\=)/2 : numeric comparison
99. (>)/2 : numeric comparison
100.      (>=)/2 : numeric comparison
101.      >>/3 : right shifts arg 1 by arg 2 bits
102. EXAMPLE (S) :
103. ?->>(16,2,_A).
104. _A = 4;
105.
106. no
107.      @/3 : alternative form for C/3 DCG connect relation
108.      (@<)/2 : instance of compare/3 with arg 1: <
109.      (@<=)/2 : instance of compare/3 with arg 1: = or <
110.      (@>)/2 : instance of compare/3 with arg 1: >
111.      (@>=)/2 : instance of compare/3 with arg 1: > or =
112.      C/3 : DCG connect predicate
113.      \\2 : complement
114.      \\3 : bitwise or of first arg with bitwise complement of second
115. EXAMPLE (S) :
116. ?-\\(0,2,_A).
117. _A = -3;
118.
119. no
120.      (+)/1 : succeeds if its argument is executed and fails
121.      V/3 : bitwise OR
122. EXAMPLE (S) :
123. ?-\\/(1,2,_A).
124. _A = 3;
125.
126. no
127.      (\\=)/2 : true if args fail to unify
128.      (\\==)/2 : true if arg 1 is not identical to arg 2
129.      ^/2 : calls arg 2 and binds arg 1
130. EXAMPLE (S) :
131. ?-_A^eq(_A,1).
132. _A = 1;
133.
134. no
135.      abolish/1 : abolish(F/N) deletes predicate F/N
136.      abort/0 : returns to toplevel
137.      abs/2 : int function
138.      abstime/1 : gets time in secs since arbitrary origin
139. EXAMPLE (S) :
140. ?-abstime(_A).
141. _A = 343416141;

```

```

142.
143. no
144.      acos/2 : float function
145.      addq/3 : adds to end of persistent queue
146. EXAMPLE (S) :
147. ?-addq(key1,key2,33) .
148.
149. yes
150.      all/2 : gets the list of terms matching arg 1 from Linda blackboard
151.      all/3 : gets a selection arg 1 of terms matching arg 2 from Linda
      blackboard
152.      all_answers/3 : (X,G,Xs): like findall/3, but such that if V is not common
      to X and G then V cannot be bound by execution of G
153. EXAMPLE (S) :
154. ?-all_answers(_A,member(s(_A),[_B,_C,_C,_D]),_E) .
155. _A = _x61091;
156. _B = s(_x62212);
157. _C = s(_x62239);
158. _D = s(_x62293);
159. _E = [_x62212,_x62239,_x62239,_x62293];
160.
161. no
162.      all_but_at_least/4 : (N,X,G,Xs) findall variant, computing all but the first
      N answers X of G
163.      all_for/2 : all_for(X,Xs) collects all constrained terms X on the
      blackboard to list Xs
164.      and/2 : conjunction, like comma
165.      answer_of/2 : (X,G): X is an answer for G, after finding all, sorting and
      removing duplicates
166.      answer_one_query/2 : handles a Jinni service S with password P
167.      append/3 : concatenates/decomposes lists
168. EXAMPLE (S) :
169. ?-append([1,2],[3,4],_A) .
170. _A = [1,2,3,4];
171.
172. no
173. ?-append(_A,_B,[1,2]) .
174. _A = [];
175. _B = [1,2];
176.
177. _A = [1];
178. _B = [2];
179.
180. _A = [1,2];
181. _B = [];
182.
183. no
184.      appendN/2 : concatenates N lists
185. EXAMPLE (S) :
186. ?-appendN([[a,b],[],[c]],_A) .
187. _A = [a,b,c];
188.
189. no

```

```

190.      append_conj/3 : concatenates 2 conjunctions
191.      append_disj/3 : concatenates 2 disjunctions
192.      apropos/1 : prints names of predicates defined in the system
193. EXAMPLE (S) :
194. ?-apropos(garbage).
195. use info/1 to get a description of a predicate
196. garbage_collect/0-[built_in,performs heap gc now]
197.
198. yes
199.      apropos/2 : returns names of predicates defined in the system
200. EXAMPLE (S) :
201. ?-apropos(retract,_A).
202. _A = retract/1-[built_in,backtracks over deleting matching
      clauses];
203.
204. _A = retract1/1-[built_in,deletes first matching clause in the
      current database];
205.
206. _A = retractall/1-[built_in,deletes all matching clauses];
207.
208. _A = x_retract/1-[built_in,db hook, backtracks over deleting
      matching clauses];
209.
210. _A = x_retractall/1-[built_in,db hook, deletes all matching
      clauses];
211.
212. _A = db_retract/2-[built_in,does retract/1 arg 2 from database
      given as arg 1];
213.
214. _A = db_retract/3-[built_in,db_retract(Db,H,B) retracts clause
      with head H and body B from database Db];
215.
216. _A = db_retract1/2-[built_in,deletes from database given as arg
      1 a matching clause];
217.
218. _A = db_retractall/2-[built_in,removes from database given as
      arg 1, all matching clauses];
219.
220. _A = db_retractall/3-[built_in,removes from database given as
      arg 1, all matching clauses seen as head + body];
221.
222. _A = prolog:x_retract_each/1-[compiled];
223.
224. _A = prolog:db_retractall0/3-[compiled];
225.
226. _A = prolog:x_retract/2-[compiled];
227.
228. _A = prolog:x_retract_args/5-[compiled];
229.
230. no
231.      arg/3 : arg(I,T,X) extracts arg I of term T to be unified with X
232. EXAMPLE (S) :
233. ?-arg(2,f(a,b),_A).
234. _A = b;
235.
236. no

```

```

237.      argn/3 : generates all n args of term
238. EXAMPLE (S) :
239. ?-argn(_A, f(a,b), _B) .
240. _A = 1;
241. _B = a;
242.
243. _A = 2;
244. _B = b;
245.
246. no
247.      arith_dif/2 : arithmetic comparison
248.      arith_eq/2 : arithmetic comparison
249.      array_get/3 : gets and dereferences array element
250.      array_get0/3 : gets array element
251.      array_set/3 : sets array element
252.      asin/2 : float function
253.      ask/5 : ask(ClientSocket,X,G,W,R): calls rpc server on ClientSocket
      with query X goal G password W and gets result R back - supports socket reuse
254.      ask_engine/2 : ask_engine(E,X) retrieves from engine E (a copy of)
      answer X
255.      ask_thread/2 : (E,R): asks an answer of engine E on a new thread R
256.      asm/0 : shows transformations and readable BinWAM assembler for
      Prolog code entered at terminal
257.      asm/1 : generates readable binarized form and BinWAM assembler to a
      file
258.      assert/1 : adds a clause
259.      assert_from_chars/1 : asserts a program from clauses in list of chars
260.      assert_from_chars/2 : (Db,Cs) asserts to database Db, a set of clauses
      parsed from list of char codes Cs
261.      asserta/1 : adds a clause to be first in a predicate definition
262.      asserted/1 : runs a predicated if asserted
263.      assertz/1 : adds a clause to be last in a predicate definition
264.      atan/2 : float function
265.      atan2/3 : float function
266.      atom/1 : true if symbol (functor of arity 0)
267. EXAMPLE (S) :
268. ?-atom(a) .
269.
270. yes
271.      atom_chars/2 : (Atom,CharAtoms): converts between an atom and its list
      of char atoms representation
272. EXAMPLE (S) :
273. ?-atom_chars(hello, _A) .
274. _A = [h,e,l,l,o];
275.
276. no
277. ?-atom_chars(_A, [104,101,108,108,111]) .
278. no
279.      atom_codes/2 : (Atom,CharCodes): converts between an atom and its list
      of char code representation

```

```

280. EXAMPLE (S) :
281. ?-atom_codes(hello,_A) .
282. _A = [104,101,108,108,111];
283.
284. no
285. ?-atom_codes(_A,[104,101,108,108,111]) .
286. _A = hello;
287.
288. no
289.      atomic/1 : true if an integer or symbolic constant
290.      bagof/3 : all solutions predicate generating unsorted bags of possibly
                    duplicated answers
291. EXAMPLE (S) :
292. ?-bagof(_A,member(_A,[3,2,2,1]),_B) .
293. _A = _x61091;
294. _B = [3,2,2,1];
295.
296. no
297.      bb/0 : lists (long!) content of the blackboard
298.      bb_def/3 : bb_def(K1,K2,T) associates to K1 and K2 (a copy of) T on the
                    blackboard
299.      bb_gc/0 : performs blackboard gc now, if enabled
300.      bb_gc0/0 : performs blackboard gc now, in this engine
301.      bb_get/3 : bb_get(K1,K2,T) consumes the term T associated with K1 and
                    K2
302.      bb_let/3 : bb_let(K1,K2,T) updates or defines the term associated with K1
                    and K2 to be T
303.      bb_reset/1 : cleans up and resizes to at least arg 1 bytes compound term
                    area of the blackboard
304.      bb_rm/2 : removes the term associated with K1 and K2 from the
                    blackboard
305.      bb_set/3 : bb_set(K1,K2,T) updates the term associated with K1 and K2
                    to be a copy of T
306.      bb_val/3 : bb_val(K1,K2,T) T is (a copy of) the term associated with keys
                    K1 and K2
307.      bbgc/0 : enables blackboard gc
308.      bbgc_status/1 : shows if blackboard gc is enabled or not
309.      begin_critical/0 : begin serialized execution - enters critical region
310.      bg/1 : runs Goal in background thread - you can set engine size like in
                    heap(500)=>bg(...)
311.      bg/2 : runs Goal in new background thread, which is returned in second
                    arg
312.      bg/3 : bg(Goal,Thread,EngineAddr): runs goal in background if threads
                    are available
313.      bg/4 : bg(Goal,Thread,EngineAddr,EngineID): runs goal in background
                    on new engine - with unique EngineID
314.      bg/7 : bg(Goal,H,S,T,-Thread,-EngineAddr,-EngineID): runs goal in
                    background on engine with given heap,stack,trail
315.      bincall/2 : call binary predicate with given continuation

```

316. **boot/0** : *regenerates file wam.bp in BinProlog src directory*
 317. **bp_info/2** : *keeps basic help info on BinProlog builtins*
 318. **bp_only/1** : *runs goal only if is_prolog(binprolog) is true*
 319. **bp_only/2** : *(Goal,Alternative): runs goal only if is_prolog(binprolog) is true otherwise runs Alternative*
 320. **bp_val/3** : *unifies with 2 key indexed global logical variable*
 321. **call/1** : *executes (atomic!) arg 1*
 322. **call/2** : *efficient call/N variant*
 323. **call/3** : *efficient call/N variant*
 324. **call/4** : *efficient call/N variant*
 325. **call/5** : *efficient call/N variant*
 326. **call/6** : *efficient call/N variant*
 327. **call/7** : *efficient call/N variant*
 328. **call_cont/1** : *calls arg 1 as current continuation*
 329. **call_external/3** : *args:*
 StringToStringFunctionAddress,InputChars,OuputChars - calls a C function
 330. **call_ifdef/2** : *calls if predicate head is defined, calls arg 2 if not*
 331. **call_jinni/1** : *call_jinni(G) calls Jinni in Twin Prolog with first answer binding G and fails if no answers are found*
 332. **call_jinni/2** : *call_jinni(G,R) calls Jinni in Twin Prolog with first answer R=the(G) or R=no, without binding G*
 333. **call_jinni/3** : *call_jinni(X,G,R) calls Jinni in Twin Prolog with first answer R=the(X) or R=no, without binding G*
 334. **callable/1** : *checks if predicate head is defined (callable)*
 335. **catch/3** : *ISO Prolog exception operator: executes arg 1 and if it catches arg 2, it executes arg 3*
 336. **cd/0** : *changes local dir to HOME directory or / if no such env var*
 337. **cd/1** : *changes local dir to arg 1*
 338. **cdelq/4** : *deletes first matching element from a queue*
 339. **EXAMPLE (S)** :
 340. `?-cdelq(key1,key2,_A,_B) .`
 341. `_A = _x61095;`
 342. `_B = 33;`
 343. `.`
 344. `no`
 345. **cdelq_any/3** : *deletes any matching element from a queue*
 346. **EXAMPLE (S)** :
 347. `?-cdelq_any(key1,key2,_A) .`
 348. `no`
 349. **ceiling/2** : *float to int function*
 350. **EXAMPLE (S)** :
 351. `?-ceiling(1.3,_A) .`
 352. `_A = 2;`
 353. `.`
 354. `no`
 355. **change_arg/3** : *destructive: change_arg(I,T,X) replaces arg I of T with X*
 356. **EXAMPLE (S)** :
 357. `?-change_arg(2,f(a,b),c) .`
 358. `.`
 359. `yes`

360. **char_of/2** : *reads a Prolog file to a set of ascii codes - on backtracking*
 361. **check_password/1** : *checks that password matches default password*
 362. **cin/1** : *tries to get and remove a term from Linda blackboard*
 363. **clause/2** : *clause(H,B) generates a clause with head matching H and body*

B

364. **clause_of/2** : *reads a Prolog file to a set of clauses - on backtracking*
 365. **clean_up_dead_engines/0** : *frees resources used by dead engines*
 366. **clean_up_engines/0** : *frees resources used by all engines except main*
 367. **clean_up_engines/1** : *internal predicate*
 368. **clear_bp_error/0** : *clears errors set by various conditions*
 369. **clone_term/3** : *clone_term(Vs,T,CT) does copy_term(T,C) while keeping unchanged variables Vs - useful if doing things like setarg/3 on the new copy*

370. **EXAMPLE (S)** :

371. `?-clone_term([_A,_C],f(_A,_B,_B,_C),_D).`

372. `_A = _x61783;`

373. `_B = _x61101;`

374. `_C = _x61786;`

375. `_D = f(_x61783,_x61784,_x61784,_x61786);`

376.

377. `no`

378. **close/1** : *closes a stream opened by open/3*

379. **close_iterator/1** : *(Iterator) closes an iterator*

380. **close_socket/1** : *closes a server, service or client*

381. **close_stream/2** : *(Type,StreamID) closes various streams*

382. **cmake/0** : *compiles BinProlog's Prolog components to compact C code - for packaging as standalone executable*

383. **cmake/1** : *compiles a Project to compact C code - for packaging as standalone executable*

384. **cmake/2** : *(Project,Module): compiles to C a project with all clauses belonging to Module*

385. **cmembq/3** : *generates (copies of) members of a queue*

386. **EXAMPLE (S)** :

387. `?-cmembq(key1,key2,_A).`

388. `no`

389. **cnl/0** : *writes a new line*

390. **co/0** : *reconsults/recompiles last file*

391. **co/1** : *reconsults using fast reader*

392. **codes_words/2** : *converts a list of character codes to a list of words and back*

393. **compare/3** : *returns <,> in arg 1 after comparing arg 2 with arg 3*

394. **EXAMPLE (S)** :

395. `?-compare(_A,1,2).`

396. `_A = (<);`

397.

398. `no`

399. `?-compare(_A,f(b),f(a)).`

400. `_A = (>);`

401.

402. `no`

403. `?-compare(_A,s(_B),s(_B)).`

```

404. _A = (=);
405. _B = _x61101;
406.
407. no
408.      compile/1 : applies current compilation method to the file arg 1
409.      compound/1 : true if it has arity > 0
410. EXAMPLE (S) :
411. ?-compound(f(a)).
412.
413. yes
414.      compute/4 : applies Op to arg 2 and arg 3 giving a result
415.      consult/1 : consults with possible duplication of clauses, allows later
      dynamic recompilation depending on db_ratio/1
416.      consult/2 : consult(File,DB) consults File into DB
417.      copy_term/2 : returns a copy of arg 1 with fresh variables
418. EXAMPLE (S) :
419. ?-copy_term(f(_A,_A,_B,_B),_C).
420. _A = _x61092;
421. _B = _x61094;
422. _C = f(_x61580,_x61580,_x61582,_x61582);
423.
424. no
425.      cos/2 : float function
426.      count_terms/2 : returns the number of terms attached to a key
427.      cout/1 : adds a term to the blackboard, unless already a matching one is
      there
428.      cpopq/3 : pops (copy of) first element of persistent queue
429. EXAMPLE (S) :
430. ?-cpopq(key1,key2,_A).
431. no
432.      create_engine/1 : makes a new engine or reuses a dead one
433.      create_engine/4 : create_engine(Heap,Stack,Trail,IntHandle) creates an
      engine IntHandle
434.      create_new_engine/1 : creates an engine as big as the current one
435.      ctime/1 : gets elapsed cpu time in ms
436. EXAMPLE (S) :
437. ?-ctime(_A).
438. _A = 265;
439.
440. no
441.      current_db/1 : gets the name of currently active database
442.      current_engine/1 : gets the unique id of the current engine
443.      current_engine_addr/1 : returns current engine handle
444.      current_engine_id/1 : returns a unique id associated to an engine at
      creation time
445.      current_engine_thread/1 : gets from the current Engine the thread it is
      running on
446.      current_input/1 : gets current input stream
447.      current_module/1 : gets name of current module
448.      current_op/3 : generates/check current op/3 operator definition(s)
449.      current_output/1 : gets current output stream

```

450. **current_predicate/1** : *generates/checks the head of an existing predicate*
 451. **current_predicate/2** : *generates/checks name and head of a currently defined predicate*
 452. **current_thread/1** : *gets thread id number of current thread*
 453. **cut_to/1** : *cuts to an int in arg 1, a choicepoint address*
 454. **cwrite/1** : *basic but quick C-version of write/1*
 455. **db_abolish/2** : *db_abolish(DB,F/N) removes predicate F/N from DB*
 456. **db_assert/2** : *does assert/1 arg 2 into database given as arg 1*
 457. **db_asserta/2** : *does asserta/1 arg 2 into database given as arg 1*
 458. **db_asserted/2** : *runs predicate arg 2 if asserted in database arg 1*
 459. **db_assertz/2** : *does assertz/1 arg 2 into database given as arg 1*
 460. **db_clause/3** : *clause(DB,H,B) generates a clause found in database DB with head matching H and body B*
 461. **db_clean/0** : *abolishes all predicates in currently active database*
 462. **db_clean/1** : *db_clean(DB) abolishes all predicates in DB*
 463. **db_consult/2** : *db_consult(File,Db) consults a file to a database Db*
 464. **db_head/2** : *generates/checks a predicate head in database (arg 1)*
 465. **db_is_dynamic/2** : *checks if dynamic in a given database*
 466. **db_listing/1** : *lists clauses given database*
 467. **db_listing/2** : *lists predicate F/N in given database*
 468. **db_move/2** : *db_move(FromDB,ToDB) moves the content of database FromDB over database ToDB while replacing similar predicates*
 469. **db_ratio/1** : *sets/gets call/assert ratio which triggers dynamic recompilation*
 470. **db_retract/2** : *does retract/1 arg 2 from database given as arg 1*
 471. **db_retract/3** : *db_retract(Db,H,B) retracts clause with head H and body B from database Db*
 472. **db_retract1/2** : *deletes from database given as arg 1 a matching clause*
 473. **db_retractall/2** : *removes from database given as arg 1, all matching clauses*
 474. **db_retractall/3** : *removes from database given as arg 1, all matching clauses seen as head + body*
 475. **db_save/1** : *db_save(File) saves all the clauses of the current database to File*
 476. **db_save/2** : *db_save(Db,File) saves using qprint/1 all the clauses of Db to File*
 477. **dcg_call/4** : *(F,X,I,O): DCG metacall for star,plus,one*
 478. **dcg_call/5** : *(F,X,Y,I,O): DCG metacall for star,plus,one*
 479. **dcg_connect/1** : *handles a terminal symbol in HAGs, as [a] in DCGs*
 480. **dcg_def/1** : *backtrackable: sets current Assumption Grammar stream - usually a hidden DCG list*
 481. **dcg_tell/1** : *switches to hidden DCG-stream number K (form 0 to MAXDCG=255)*
 482. **dcg_telling/1** : *retrieves which hidden DCG-stream we are processing*
 483. **EXAMPLE(S)** :
 484. `?-dcg_telling(_A).`
 485. `_A = 1;`

```

486.
487. no
488.      dcg_val/1 : backtrackable: retrieves current Assumption Grammar stream
      - usually a hidden DCG list
489.      dconsult/1 : reconsult/1 variant: cleans up data areas, consults/compiles
      based on db_ratio/1
490.      debugmes/1 : writes message in debug mode (low quietness)
491.      deep_hash/4 : (Key,Depth,Mod) computes hashvalue of Key modulo Mod,
      up to max recursion Depth
492. EXAMPLE (S) :
493. ?-deep_hash(f(a),5,0,_A) .
494. _A = 23114257;
495.
496. no
497. ?-deep_hash(f(b),1,32,_A) .
498. _A = 18;
499.
500. no
501.      default/2 : default that can be overridden if asserted
502. EXAMPLE (S) :
503. ?-default(host(_A),_A = localhost) .
504. _A = localhost;
505.
506. no
507.      default_host/1 : returns default host for remote Linda server
508.      default_login/1 : returns default (nick)name for user
509.      default_password/1 : returns default password for user
510.      default_port/1 : returns default port for remote Linda server
511.      default_this_host/1 : returns default IP address or hostname this machine
512.      default_this_port/1 : returns default port to work as a server on
513.      default_timeout/1 : returns default timeout
514.      delete_all_terms/1 : (GroundKey) removes all terms attached to a key
515.      destroy_array/1 : frees an array
516.      destroy_engine/1 : destroy_engine(E) frees memory of engine E (an
      integer)
517.      det_call/1 : calls a Goal and warns if it was not deterministic
518.      detect_ip_addr/1 : the ip address of current host, if detected, that of
      localhost if not
519.      detect_user/1 : guesses the user from environment information
520.      dir/0 : lists files under DOS
521.      dir2dirs/2 : (Dir,Dirs): converts dir cmd output to list of sub directories
      of Dir
522.      dir2files/2 : (Dir,Files): converts dir cmd output to list of files (which are
      not dirs) contained in Dir
523.      dir2list/3 : (DirListerCmd,DirName,Files): converts OS specific DirLister
      output to list of files and/or directories
524.      (discontiguous)/1 : states that clauses of a predicate can be in different
      places
525.      display/1 : writes to terminal while ignoring operator definitions

```

- 526. **drop_at_least/2** : *(N,Goal) drops at least N answers of Goal G*
- 527. **dyn2stat/1** : *dyn2stat(H): compiles at runtime a predicate with head H*
- 528. **dyn2stat/2** : *dyn2stat(Db,H): compiles at a predicate from database Db
with head H*
- 529. **(dynamic)/1** : *states that a predicate can be updated*
- 530. **dynbbgc/0** : *makes blackboard dynamic with gc on*
- 531. **dynco/1** : *yes/no: activates/desactivates dynamic recompilation*
- 532. **ed/0** : *edits last compiled/consulted file with default editor and refreshes it
in memory*
- 533. **edit/0** : *calls DOS editor edit on last compiled file*
- 534. **edit/2** : *edit(E,F) edits with editor E, file F*
- 535. **element_of/2** : *(Engine,Answer: backtracks over the answers of a fluent
(usually an engine)*
- 536. **emacs/0** : *calls emacs editor on last compiled file*
- 537. **end_critical/0** : *ends serialized execution - exits critical region*
- 538. **end_module/0** : *ends current module*
- 539. **end_module/1** : *ends module if current, signals error if not*
- 540. **end_of_file/0** : *Prolog atom returned by read when at the end of a file*
- 541. **eq/2** : *unifies arg 1 and arg 2, like =*
- 542. **errmes/2** : *writes error message and fails*
- 543. **exists_file/1** : *true if file exists*
- 544. **exit/0** : *same as halt*
- 545. **exp/2** : *float function*
- 546. **expand_term/2** : *expands a term according to DCG expansion rules*
- 547. **fail/0** : *always fails*
- 548. **false/0** : *always fails*
- 549. **fatal_error/2** : *writes error message and aborts*
- 550. **fcall/3** : *calls a list to list function: to be implemented*
- 551. **fclose/1** : *closes the C-stream specifying as an integer handle*
- 552. **fcompile/1** : *compiles a *.pl file to a *.wam bytecode file*
- 553. **fflush/1** : *fflush(IntegerStreamNo) flushes a C-stream*
- 554. **fgetc/2** : *fgetc(IntegerStreamNo,CharCode) inputs a char code from a C
stream*
- 555. **file2chars/2** : *reads a file to a list of ascii codes*
- 556. **file2sock/2** : *writes to a socket, from a file*
- 557. **file_cmd_hook/2** : *file_cmd_hook(Cmd,Db): allows defining user actions
on commands read from files*
- 558. **file_extension_list/1** : *defines default file extensions for find_file*
- 559. **file_search_path/1** : *defines search path relative to BP_PATH (home of
BinProlog) and PROLOG_PATH (home of user files)*
- 560. **file_size/2** : *returns the size of a file, in bytes*
- 561. **find_at_most/4** : *(N,X,G,Xs) findall variant, computing at most N answers
X of G*
- 562. **find_file/2** : *finds a file name on search path*
- 563. **find_while/4** : *findall variant*

564. findall/3 : *findall(X,G,Xs) collects copies of all answers X of G to Xs. If less than half of the heap is free, it allocates new engine for running G*

565. EXAMPLE (S) :

566. `?-findall(s(_A), (member(_A, [1,2,3]), _A > 1), _B).`

567. `_A = _x61095;`

568. `_B = [s(2), s(3)];`

569.

570. `no`

571. findall/4 : *findall(X,G,Xs,Ys) appends the list of answers X of G to Ys to obtain Xs*

572. EXAMPLE (S) :

573. `?-findall(s(_A), (_A = 1 ; _A = 2), _B, [3,4]).`

574. `_A = _x61102;`

575. `_B = [s(1), s(2), 3, 4];`

576.

577. `no`

578. float/1 : *true if represented as a 64 bit float number (C-double)*

579. EXAMPLE (S) :

580. `?-float(3.14).`

581.

582. `yes`

583. float/2 : *float function*

584. floor/2 : *float to int function*

585. EXAMPLE (S) :

586. `?-floor(1.3, _A).`

587. `_A = 1;`

588.

589. `no`

590. flush_output/0 : *flushes current output stream*

591. flush_output/1 : *flushes a stream*

592. foldl/4 : *(Op,InitialValue,List,?Result) accumulates values iterating over List with binary Op*

593. EXAMPLE (S) :

594. `?-foldl(+, 0, [10,20,30], _A).`

595. `_A = 60;`

596.

597. `no`

598. foldr/4 : *(Op,InitialValue,List,?Result) accumulates values iterating over List with binary Op*

599. EXAMPLE (S) :

600. `?-foldr(+, 0, [10,20,30], _A).`

601. `_A = 60;`

602.

603. `no`

604. fopen/3 : *Prolog equivalent of C-function: opens a stream in a given mode and returns an integer handle to it*

605. for/3 : *generates an integer in a range*

606. EXAMPLE (S) :

607. `?-for(_A, 1, 3).`

608. `_A = 1;`

609.

610. `_A = 2;`

611.

```

612. _A = 3;
613.
614. no
615. for_all/2 : foreach(A,B) fails for all cases when A succeeds and B fails
616. forall/1 : forall(G) backtracks over all answers to G and succeeds
617. forall/2 : forall(G,D) executes D once for each answer of generator G
618. foreach/1 : foreach(G) backtracks over all answers to G and succeeds
619. foreach/2 : foreach(G,D) executes D once for each answer of generator G
620. fputc/2 : fputc(IntegerStreamNo,CharCode) outputs a char code to a C
    stream
621. free_term/1 : (Handle): frees external term given as a Handle
622. free_thread_guard/1 : gives back to pool thread guard in arg 1
623. from_engine/1 : (T) tries to take a term T from the message box of this
    engine and sets the message box empty
624. fsize/2 : returns the size of the file associated to a C stream, in bytes
625. functor/3 : builds or decomposes a compound term
626. EXAMPLE (S) :
627. ?-functor(f(a,b),_A,_B) .
628. _A = f;
629. _B = 2;
630.
631. no
632. ?-functor(_A,f,3) .
633. _A = f(_x61710,_x61711,_x61712);
634.
635. no
636. ?-functor(f(a),f,1) .
637.
638. yes
639. garbage_collect/0 : performs heap gc now
640. gc/0 : enables heap gc
641. gc_call/1 : G: executes G and ensures that no more space is consumed
    than the total size of the terms bound to its variables
642. gc_status/1 : shows if heap gc is enabled or not
643. generic_write/1 : overridable write/1, style (writeq, write, display) given
    with write_style/1 assumption
644. gensym/2 : generates a new name based on arg 1
645. gensym_no/2 : generates a new number based on arg 1
646. get/1 : inputs the next char code after skiping over white space
647. get/2 : get(E,A) returns a new answer A=the(...) from engine E or returns
    no if no (more) answers exist
648. get0/1 : reads a char as an ascii code
649. get_all_terms/2 : (GroundKey,Ts) collects to alist all terms attached to a
    key
650. get_bp_error/4 : (Id,Mes,Arg1,Arg2): gets error sate from emulator
651. get_char/2 : (Stream,CharAsOneLetterConstant): inputs a char from a
    stream -ISO Prolog
652. get_code/1 : ISO char code reader
653. get_code/2 : inputs a char code from a stream - ISO

```

654. **get_cont/1** : *captures current continuation, usually an cyclic term*
 655. **get_critical/2** : *(Guard,Data): accesses mutex Guard protected Data on
blackboard*
 656. **get_deep_cut/2** : *gets a choice point address, used with 1 arg only*
 657. **get_engine_id/2** : *(+Engine,-Id) gets the unique id associated to an
engine at creation time*
 658. **get_engine_prop/3** : *args: Engine,PropertyNo,Val*
 659. **get_engine_thread/2** : *gets from an Engine the thread it is running on*
 660. **get_lineno/1** : *gets line number counter in current file*
 661. **get_load_method/1** : *gets the current load method by name*
 662. **get_neck_cut/1** : *gets the choice point as an integer*
 663. **get_next_term/2** : *(Iterator,Term) returns the next term attached to an
iterator, fails if no more left*
 664. **get_password/1** : *gets default password for user*
 665. **get_term/2** : *(Iterator,Term): backtracks over terms associated to a key*
 666. **global_get/3** : *(A,B,X): retrieves X associated to ground keys A,B*
 667. **global_rm/2** : *(A,B): removes value associated to ground keys A,B*
 668. **global_set/3** : *(A,B,X): associates X to ground keys A,B*
 669. **greater/2** : *arithmetic comparison*
 670. **greater_eq/2** : *arithmetic comparison*
 671. **ground/1** : *true if arg has no free variables*
 672. **EXAMPLE (S)** :
 673. ?-ground(f(a,b)).
 674.
 675. yes
 676. **halt/0** : *stops BinProlog*
 677. **halt/1** : *stops Prolog with given return code when used in main thread- or
halts current thread*
 678. **handle_service/2** : *handles a Jinni service S with password P - always
succeeds*
 679. **has_info/1** : *checks/generates predicates Pred/Arity for which info is
available*
 680. **has_terms/1** : *(GroundKey) succeeds if at least on term is attached to
GroundKey*
 681. **has_threads/0** : *succeeds if threads available on this platform*
 682. **hash_clear/0** : *removes all terms from all hash keys*
 683. **hash_clear/1** : *(GroundKey) removes all terms attached to a key*
 684. **hash_clear/2** : *(GroundKey,Term) removes all matching terms attached to
a key*
 685. **hash_compile/0** : *compiles the hashed map to a predicate benefiting from
1-arg indexing*
 686. **hash_find_unique/3** : *like findall(X,G,Xs) but uses hashing on ground
term X to collect unique answers*
 687. **hash_gc/0** : *frees space used by hash keys and values*
 688. **hash_get/2** : *(GroundKey,Term) retrieves in constant time a term attached
to a ground key*
 689. **hash_key/1** : *(IntKey) backtrack over hash keys*

690. **hash_load/1** : *(File) loads the hashed map to a database*
 691. **hash_push/2** : *(GroundKey,Term): attaches a term to a ground key in constant time as a the first element*
 692. **hash_put/2** : *(GroundKey,Term): attaches a term to a ground key in constant time as the last element*
 693. **hash_rm/2** : *(GroundKey) removes a term attached to a key and returns it - backtracks*
 694. **hash_rm_one/2** : *(GroundKey) removes a term attached to a key and returns it*
 695. **hash_save/1** : *(File) saves the hashed map to a database ready for 1-arg indexing if compiled*
 696. **hash_save/2** : *(File,PredNMame) saves the hashed map to a database in predicate PredName ready for 1-arg indexing if compiled*
 697. **hash_trace/2** : *tracer for hash maps*
 698. **help/0** : *generates file help.txt with info and examples*
 699. **help/1** : *same as apropos/1*
 700. **EXAMPLE(S)** :
 701. ?-help(assert).
 702. use info/1 to get a description of a predicate
 703. assert/1-[built_in,adds a clause]
 704. assert_from_chars/1-[built_in,asserts a program from clauses in list of chars]
 705. assert_from_chars/2-[built_in,(Db,Cs) asserts to database Db, a set of clauses parsed from list of char codes Cs]
 706. asserta/1-[built_in,adds a clause to be first in a predicate definition]
 707. asserted/1-[built_in,runs a predicated if asserted]
 708. assertz/1-[built_in,adds a clause to be last in a predicate definition]
 709. db_assert/2-[built_in,does assert/1 arg 2 into database given as arg 1]
 710. db_asserta/2-[built_in,does asserta/1 arg 2 into database given as arg 1]
 711. db_asserted/2-[built_in,runs predicate arg 2 if asserted in database arg 1]
 712. db_assertz/2-[built_in,does assertz/1 arg 2 into database given as arg 1]
 713. get_asserted/2-[built_in]
 714. is_asserted/1-[built_in,checks if currently asserted]
 715. prolog:assert_it/2-[compiled]
 716. prolog:db_is_asserted/2-[compiled]
 717. prolog:is_asserted_clause/1-[compiled]
 718. prolog:x_assert/2-[compiled]
 719. prolog:x_assert_op/3-[compiled]
 720. x_asserta/1-[built_in,db hook, adds a clause to be first in a predicate definition]
 721. x_assertz/1-[built_in,db hook, adds a clause to be last in a predicate definition]
 722.
 723. yes
 724. **hide_default/1** : *makes unavailable a default value*
 725. **hkey/2** : *computes hash code on atomic argument; fails on variables and compound terms*

```

726. EXAMPLE (S) :
727. ?-hkey(t(a,b),_A) .
728. _A = 236945626;
729.
730. no
731. ?-hkey(t(a,c),_A) .
732. _A = 236945625;
733.
734. no
735. host/1 : assumes default host for Linda server
736. host2ip/2 : converts a host name to an IP address
737. hostname/1 : the name of current host, if detected, localhost if not
738. hypot/3 : float function
739. if_any/3 : (Cond,Then,Else): executes Cond; each time when Cond
   succeeds it also executes Then; if Cond never succeeds it executes Else
740. in/1 : waits to remove a term from Linda blackboard
741. include/1 : includes/loads a file with current load method, unless it has
   already been included by the same method
742. info/0 : generates info on predicates with examples
743. info/1 : generates info and examples of use for predicate Pred/Arity
744. init_gensym/1 : resets gensym for names based on arg 1
745. init_io/0 : intialises fast C-based IO routines
746. instance_of/2 : (Handle,Term): creates an internal instance of an external
   term given as a Handle
747. integer/1 : true if an integer
748. integer/2 : float to int cast
749. interactive/1 : toggles interactive query answering/tracing with arg 1 =
   yes or no
750. (is)/2 : calls the function evaluator, mostly for arithmetics
751. EXAMPLE (S) :
752. ?-_A is 3+4*2.
753. _A = 11;
754.
755. no
756. is_an/1 : true if an alphanumerical char code
757. is_asserted/1 : checks if currently asserted
758. is_builtin/1 : recognizes a predicate head as a builtin
759. EXAMPLE (S) :
760. ?-is_builtin(var(_A)) .
761. _A = _x61088;
762.
763. no
764. is_compiled/1 : true if head of a compiled predicate
765. is_discontiguous/1 : checks if a predicate has been declared as
   discontiguous
766. is_dynamic/1 : checks if dynamic
767. is_engine/1 : recognizes and integer as an engine handle
768. is_interactive/0 : checks if toplevel is interactive - use interactive/1 with
   yes/no to set it the way you want it
769. is_maj/1 : true if is an upper case char code

```

770. **is_min/1** : *true if a lower case char code*
 771. **is_module/1** : *recognizes/generates a module name*
 772. **is_num/1** : *true if a digit char code*
 773. **is_prolog/1** : *recognizes binprolog - useful for portability*
 774. **is_public/1** : *checks predicate head if globally visible*
 775. **is_spec/1** : *true if a spacial character code*
 776. **is_terminator/1** : *true if a terminator character code*
 777. **keygroup/3** : *sorts while grouping similar keys*
 778. **EXAMPLE (S)** :
 779. `?-keygroup([3-a,1-a,2-b,1-c,2-d],_A,_B).`
 780. `_A = 1;`
 781. `_B = [a,c];`
 782.
 783. `_A = 2;`
 784. `_B = [b,d];`
 785.
 786. `_A = 3;`
 787. `_B = [a];`
 788.
 789. `no`
 790. **keysort/2** : *sorts while putting similar keys one after the other in a list*
 791. **EXAMPLE (S)** :
 792. `?-keysort([3-a,1-a,2-b,1-c,2-d],_A).`
 793. `_A = [1-a,1-c,2-b,2-d,3-a];`
 794.
 795. `no`
 796. **length/2** : *generates/mesures length of a list*
 797. **less/2** : *arithmetic comparison*
 798. **less_eq/2** : *arithmetic comparison*
 799. **line_of/2** : *(File,Line) reads a file to lines ending with eol - on backtracking*
 800. **list_engines/0** : *prints out the list of active engines*
 801. **list_engines/1** : *lists available engine handles*
 802. **listify/2** : *(T,Cs): transforms T, unless it is already such, to list of chars*
 803. **listing/0** : *lists current database*
 804. **listing/1** : *lists given predicate if in current database*
 805. **listing/2** : *lists predicate F of arity N if in current database*
 806. **load/1** : *clean loads from a bytecode *.wam file*
 807. **load_engine/3** : *load_engine(E,Goal,Answer) prepares engine E to execute Goal/Answer*
 808. **load_method/2** : *args: Number, Name*
 809. **EXAMPLE (S)** :
 810. `?-load_method(_A,_B).`
 811. `_A = 1;`
 812. `_B = mcompile;`
 813.
 814. `_A = 2;`
 815. `_B = scompile;`
 816.
 817. `_A = 3;`
 818. `_B = oconsult;`

```

819.
820. _A = 4;
821. _B = dconsult;
822.
823. _A = 5;
824. _B = sconsult;
825.
826. _A = 6;
827. _B = load;
828.
829. no
830. local_all/2 : local_all(X,Xs) collects all facts matching X on the
blackboard
831. local_all/3 : local_all(X,G,Xs) collects facts X such that G on the
blackboard
832. local_cin/1 : removes a term if available on the local blackboard
833. local_cout/1 : puts a term on local blackboard unless already there
834. local_in/1 : waits for a term produced by a matching local_out/1
835. local_out/1 : produces a term and possibly wakes up the thread at a
matching local_in/1
836. local_rd/1 : tests if a term is available on the local blackboard
837. local_when/1 : waits until a term is available on the local blackboard
838. lock_thread_guard/1 : locks thread guard created with
new_thread_guard
839. log/2 : float function
840. log/3 : returns log in base arg 1 of arg 2, a float
841. EXAMPLE (S) :
842. ?-log(2,8,_A) .
843. _A = 3;
844.
845. no
846. login/1 : assumes default (nick)name for user
847. ls/0 : list files under Unix
848. ls2list/2 : (Dir,Files): converts ls cmd output to list of files and dirs
849. lval/3 : backtrackable: lval(K1,K2,V) associates V to keys K1 and K2
850. EXAMPLE (S) :
851. ?-lval(a,b,f(_A)) .
852. _A = _x611110;
853.
854. no
855. main/0 : user defined optional startup predicate
856. main/1 : default entry predicate, does both prolog_init/1 and
prolog_run/1
857. make_array/2 : creates an array
858. make_cmd/2 : concatenates a list of strings and atomic elements into an
atom
859. make_cmd0/2 : concatenates a list of strings and atomic elements into a
string
860. make_compileable/2 : make_compileable(Files,File) rewrites Files as a
canonical File with all predicates contiguous and no operators

```

861. **make_spaced_cmd/2** : concatenates a list of elements with inserted space
 separators into an atom

862. **map/2** : maps a predicate with 1 arg to a list

863. **EXAMPLE (S)** :

864. ?-map(println,[10,20,30]).

865. 10

866. 20

867. 30

868. .

869. yes

870. **map/3** : maps a predicate with 2 args to a list

871. **EXAMPLE (S)** :

872. ?-map(+1,[10,20],_A).

873. _A = [11,21];

874. .

875. no

876. **map_clear/1** : (D): empties and cleans up D

877. **map_gc/1** : (D): frees unused memory in D

878. **map_get/3** : (D,K,V): gets V, given K in map D, backtracks over K if K
 unbound

879. **map_key/2** : (D,K): succeeds if K is known, otherwise finds V backtracks
 over each key K in D

880. **map_new/1** : (D): returns a new map handle D

881. **map_put/3** : (D,K,V): adds K,V to map D

882. **map_rm/2** : (D,K): removes key K from map D

883. **map_show/1** : (D): lists the content of a map

884. **maps_clean/0** : empties and cleans up all maps created with mmap_new

885. **maps_gc/0** : frees memory for all maps created with mmap_new

886. **maps_iterate/1** : (D): backtracks over map handles created with
 mmap_new D

887. **maps_show/0** : lists the content of all maps created with mmap_new

888. **match_before/4** : match_before(+Stop,-Word,I,O): matches a word with
 DCGs until a given delimiter is hit

889. **match_before/5** : match_before(+Stops,-Word,-Stop,I,O): matches a
 word with DCGs until a given delimiter set is hit, one of which is also returned

890. **match_word/3** : match_word(+Word,I,O): matches/consumes a word, in
 the context of DCGs

891. **max/3** : (X,Y,Max): Max is the max of 2 numbers X, Y

892. **mcompile/1** : compile/1 variant: cleans up data areas and compiles to
 memory

893. **member/2** : (X,Xs): checks if an element X unifies with an element on a
 list Xs or generates successively longer lists if Xs is unbound or open ended

894. **EXAMPLE (S)** :

895. ?-member(2,[1,2]).

896. .

897. yes

898. ?-member(_A,[1,2]).

899. _A = 1;

900. .

901. _A = 2;

```

902.
903. no
904.      member_conj/2 : like member/2, for a comma separated conjunction,
      ending with true
905. EXAMPLE (S) :
906. ?-member_conj(_A, (a,b,true)) .
907. _A = a;
908.
909. _A = b;
910.
911. no
912.      member_scan/3 : finds first element without unifying to it
913. EXAMPLE (S) :
914. ?-member_scan(s(_A), [1,s(s(_B))],2],_C) .
915. _A = _x61111;
916. _B = _x61121;
917. _C = [s(s(_x61121)),2];
918.
919. no
920.      memberchk/2 : (X,Xs) checks if an X is a the list Xs
921.      merge_sort/3 : (Order,List,Sorted)
922. EXAMPLE (S) :
923. ?-merge_sort(>, [1,3,2,2,4],_A) .
924. _A = [4,3,2,2,1];
925.
926. no
927.      metacall/1 : calls the interpreter
928.      metatrue/0 : calls the interpreter on current continuation
929.      min/3 : (X,Y,Min): Min is the min of 2 numbers X, Y
930.      mmap_clear/1 : (D): empties and cleans up D
931.      mmap_gc/1 : (D): frees unused memory in D
932.      mmap_get/3 : (D,K,V): gets V, given K in mmap D, backtracks over K if
      K unbound
933.      mmap_key/2 : (D,K): succeeds if K is known, otherwise finds V backtracks
      over each key K in D
934.      mmap_new/1 : (D): returns a new mmap handle D - a mmap holds
      multiple elements for each key
935.      mmap_push/3 : (D,K,V): adds K,V as the first element of mmap D
936.      mmap_put/3 : (D,K,V): adds K,V as the last element of mmap D
937.      mmap_rm/3 : (D,K,X): removes a term X attached to key K from mmap D
      and backtracks on each such term
938.      mmap_rm_all/2 : (D,K): removes all terms X attached to key K from
      mmap D
939.      mmap_rm_one/3 : (D,K,X): removes first term X attached to key K from
      mmap D
940.      mmap_show/1 : (D): lists the content of a mmap
941.      mmaps_clean/0 : empties and cleans up all mmaps created with
      mmap_new
942.      mmaps_gc/0 : frees memory for all mmaps created with mmap_new

```

943. **mmaps_iterate/1** : *(D): backtracks over mmap handles created with mmap_new D*

944. **mmaps_show/0** : *lists the content of all mmaps created with mmap_new*

945. **mod/3** : *modulo*

946. **EXAMPLE (S)** :

947. `?-mod(10,3,_A) .`

948. `_A = 1;`

949. `.`

950. `no`

951. **(mode)/1** : *accepts mode declarations although we are not using them currently*

952. **(module)/1** : *starts a module*

953. **(module)/2** : *starts a module specifying a list of visible predicates*

954. **modules/1** : *returns a list of existing modules*

955. **msort/2** : *sorts and keeps duplicates*

956. **EXAMPLE (S)** :

957. `?-msort([2,1,3,1,4,4,2],_A) .`

958. `_A = [1,1,2,2,3,4,4];`

959. `.`

960. `no`

961. **(multifile)/1** : *states that clauses of a predicate can be in different files*

962. **name/2** : *bidirectional: converts atomic to/from list of chars*

963. **EXAMPLE (S)** :

964. `?-name(hello,_A) .`

965. `_A = [104,101,108,108,111];`

966. `.`

967. `no`

968. `?-name(_A,[98,121,101]) .`

969. `_A = bye;`

970. `.`

971. `no`

972. **namecat/4** : *concatenates 3 names*

973. **EXAMPLE (S)** :

974. `?-namecat(a,:,b,_A) .`

975. `_A = a:b;`

976. `.`

977. `no`

978. **near_match/2** : *matches 2 lists of chars*

979. **new_builtin/3** : *sample 3 arg user added builtin - used for the C interface*

980. **new_client/3** : *from a (host,port) to a client*

981. **new_engine/3** : *new_engine(X,G,E) creates an engine E ready to execute goal G with answer X*

982. **new_iterator/2** : *(GroundKey,Iterator) gets an iterator to terms attached to ground key*

983. **new_key_iterator/1** : *(Iterator): returns an iterator over the set of external keys*

984. **new_name/2** : *returns a new name based on arg 1*

985. **new_server/2** : *opens on a port a new server*

986. **new_service/3** : *from (server,timeout) to a service*

987. **new_term/2** : *(Term,Handle): creates a new external Term and returns an integer Handle to it*

988. **new_thread_guard/1** : *returns a new free thread guard from pool*
 989. **nl/0** : *writes a new line character*
 990. **nobbgc/0** : *disables blackboard gc*
 991. **nogc/0** : *disables heap gc*
 992. **nonvar/1** : *true if currently instantiated*
 993. **(nospy)/1** : *do not spy on Pred/Arity anymore*
 994. **(not)/1** : *sound negation*
 995. **notepad/0** : *calls notepad editor on last compiled file*
 996. **notify_about/1** : *notifies a suspended matching wait_for(Term,Constraint),
if Constraint holds, that Term is available*
 997. **notrace/0** : *do not trace predicates when interpreted*
 998. **nth_answer/2** : *(N,Goal) returns only the nth answer of Goal, if such an
answer exists*
 999. **nth_member/3** : *retrieves N-th element of a list*
 1000. **EXAMPLE (S)** :
 1001. `?-nth_member(_A, [a,b,c], _B) .`
 1002. `_A = a;`
 1003. `_B = 1;`
 1004.
 1005. `_A = b;`
 1006. `_B = 2;`
 1007.
 1008. `_A = c;`
 1009. `_B = 3;`
 1010.
 1011. `no`
 1012. **number/1** : *true if integer or float*
 1013. **number_chars/2** : *(Number,CharAtoms): converts between a number and
its list of char atoms representation*
 1014. **EXAMPLE (S)** :
 1015. `?-number_chars(1999, _A) .`
 1016. `_A = [1,9,9,9];`
 1017.
 1018. `no`
 1019. **number_codes/2** : *(Number,CharCodes): converts between a number and
its list of char code representation*
 1020. **EXAMPLE (S)** :
 1021. `?-number_codes(1999, _A) .`
 1022. `_A = [49,57,57,57];`
 1023.
 1024. `no`
 1025. `?-number_codes(_A, [50,48,48,49]) .`
 1026. `_A = 2001;`
 1027.
 1028. `no`
 1029. **numbervars/3** : *binds to \$VAR(I) with I over distinct integers variables in
a term*
 1030. **oconsult/1** : *reconsult/1 variant: consults/compiles based on db_ratio and
overwrites old clauses*
 1031. **older_file/2** : *true if arg 1 is a file older than arg 2*

1032. **on_undefined/1** : *(Goal): defines handler for undefined predicates matching Goal*

1033. **once/1** : *executes once, with no backtracking*

1034. **one/4** : *one(Recognizer,Result): DCG based regexp processor for exactly one Recognizer*

1035. **op/3** : *op(Pri,A,Op) defines an operator Op of priority Pri and associativity A*

1036. **open/3** : *returns a stream (arg 3) on a file (arg 1) in read/write/append mode (arg 2)*

1037. **open_engine/3** : *open_engine(G,X,E) creates an engine E ready to execute goal G with answer X*

1038. **open_stream/4** : *(Type,FileOrCmd,ReadWriteAppend,?StreamID) opens various streams*

1039. **otherwise/0** : *always succeeds*

1040. **otime/1** : *gets time in secs from arbitrary origin to start*

1041. **EXAMPLE (S)** :

1042. ?-otime(_A).

1043. _A = 343416141;

1044.

1045. no

1046. **out/1** : *puts a term on Linda server or triggers resumption of a matching in/1 waiting for this data*

1047. **override/3** : *overrides a compiled predicate - to be used with care*

1048. **password/1** : *assumes default password for user*

1049. **pc/0** : *pushes code compiled into the workspace to the persistent kernel*

1050. **pclose/1** : *closes a pipe generated stream*

1051. **pcollect/2** : *collects output from a command to a list of char codes*

1052. **peer_addr/2** : *gets address of peer connected to socket*

1053. **peer_port/2** : *gets port of peer connected to socket*

1054. **phrase/2** : *(Axiom, ?InputChars): DCG evaluator, starting from Axiom, consuming/producing InputChars*

1055. **phrase/3** : *(Axiom, ?InputChars, ?OutputChars): DCG evaluator, starting from Axiom*

1056. **EXAMPLE (S)** :

1057. ?-phrase([a],[b]),[a,b|_A],_A).

1058. _A = _x61093;

1059.

1060. no

1061. **pico/0** : *calls pico editor on last compiled file*

1062. **plus/4** : *plus(Recognizer,Result): DCG based plus regexp processor for (Recognizer)+*

1063. **popen/3** : *popen(Cmd,read/write,Stream) opens Stream using a pipe from/to process executing Cmd*

1064. **port/1** : *assumes default port for Linda server*

1065. **portray_clause/1** : *pretty prints a clause*

1066. **pow/3** : *(Base,Expo,Val) computes power function*

1067. **EXAMPLE (S)** :

1068. ?-pow(2,3,_A).

1069. _A = 8;

```

1070.
1071.      no
1072.  pp_clause/1 : prints out a clause with some care on how it looks
1073.  pp_term/1 : pretty prints a term
1074.  predicate_property/2 : returns a property of a predicate
1075.      EXAMPLE (S) :
1076.      ?-predicate_property(write(_A),_B) .
1077.      _A = _x61092;
1078.      _B = built_in;
1079.
1080.      no
1081.  print/1 : variant of write/1
1082.  println/1 : synchronized printing of a term on a line
1083.  process_term/3 : (OpCode,Input,Output): applies various user defined C
functions to external terms
1084.  prod/2 : (List, ?Result): product of a list
1085.      EXAMPLE (S) :
1086.      ?-prod([10,20],_A) .
1087.      _A = 200;
1088.
1089.      no
1090.  profile/0 : in specially compiled profiler mode prints out info accumulated
so far
1091.  prolog_flag/2 : (Flag,Value): retrieves the value of a Prolog flag
1092.  prolog_init/1 : runs more command line args representing prolog goals
1093.  prolog_load/1 : loads code and/or executes first command line arg
1094.  prolog_run/1 : starts main/0 if defined, otherwise toplevel/0
1095.  (public)/1 : declares globally visible predicate Name/Arity
1096.  push_code/1 : moves code compiled in workspace to kernel and returns
top of code area - used by pc/0
1097.  push_term/2 : (GroundKey,Term): attaches a term to a ground key in
constant time as a the first element
1098.  pushq/3 : adds to beginning of persistent queue
1099.      EXAMPLE (S) :
1100.      ?-pushq(key1,key2,f(_A,_A)) .
1101.      _A = _x61094;
1102.
1103.      no
1104.  put/1 : writes and ascii code as a char
1105.      EXAMPLE (S) :
1106.      ?-put(99) .
1107.      c
1108.      yes
1109.  put_char/2 : (Stream,CharAsOneLetterConstant): outputs a char to a
stream -ISO Prolog
1110.  put_code/1 : ISO char code writer
1111.      EXAMPLE (S) :
1112.      ?-put_code(99) .
1113.      c
1114.      yes
1115.  put_code/2 : outputs a char code to a stream - ISO

```

1116. **put_critical/2** : (*Guard,Data*): updates mutex Guard-protected Data on blackboard

1117. **put_term/2** : (*GroundKey,Term*): attaches a term to a ground key in constant time as the last element

1118. **pwd/0** : shows current dir

1119. **pwd/1** : returns current dir as a list of chars

1120. **qcompile/1** : compile/1 variant: compiles a file to memory after pushing current user code to kernel - where it becomes read only

1121. **qmake/1** : compiles Project to fast C code - for packaging as standalone executable

1122. **qmake/2** : (*Project,Module*): compiles to fast C code a project in Module: uses *set_threshold(12,60)* before doing *cmake/2*, for a good speed/code size ratio

1123. **qprint/1** : prints out a clause such that a variant of it can be always read back

1124. **quiet/1** : gets/sets level of "quietness"

1125. **quietmes/1** : writes message if in low quietness mode

1126. **quietmes/2** : writes message if quietness is lower than arg 1)

1127. **quit/0** : same as halt

1128. **random/1** : returns a random integer

1129. **EXAMPLE (S)** :

1130. ?-random(_A).

1131. _A = 1457;

1132. no

1133. no

1134. **random_seed/1** : initializes random/1 with an integer, uses clock if 0

1135. **rd/1** : reads a term matching arg 1 from Linda blackboard

1136. **read/1** : reads a term

1137. **read_chars/1** : reads to a list of ascii codes

1138. **read_line/1** : reads a line into a constant

1139. **read_term/2** : reads a term and also a list of variable-name associations

1140. **read_term_from_chars/2** : reads a term from a list of char codes

1141. **EXAMPLE (S)** :

1142. ?-
 read_term_from_chars([102,40,88,44,88,44,89,44,89,41],_A).

1143. _A = f(_x62944,_x62944,_x62995,_x62995);

1144. no

1145. no

1146. **read_term_from_chars/3** : reads a term with variable names from a list of char codes

1147. **EXAMPLE (S)** :

1148. ?-
 read_term_from_chars([102,40,88,44,88,44,89,44,89,41],_A,_B).

1149. _A = f(_x63012,_x63012,_x63063,_x63063);

1150. _B = [X = _x63012,Y = _x63063];

1151. no

1152. no

1153. **read_terms_from_chars/2** : (*Chars,Clause*): backtracks over Clause(s) parsed from a list of char codes

1154. read_terms_from_chars/3 : *(Chars,Clause,VarsInClause): backtracks over Clause(s) parsed from a list of char codes*

1155. **EXAMPLE (S)** :

1156. ?-
 read_terms_from_chars([98,40,88,44,89,41,58,45,97,40,89,44,88,41,
 46,32,97,40,49,44,49,41,46,32,97,40,95,44,50,41,46,32],_A,_B) .

1157. _A = (b(_x64809,_x64837) :- a(_x64837,_x64809));

1158. _B = [X = _x64809,Y = _x64837];

1159.

1160. _A = a(1,1);

1161. _B = [];

1162.

1163. _A = a(_x64408,2);

1164. _B = [_ = _x64408];

1165.

1166. no

1167. read_words/1 : *reads a line into a list of words*

1168. reboot/0 : *regenerates BinProlog from its sources*

1169. reconsult/1 : *applies current consult method to file given as arg 1, set db_ratio/1 for specifying dynamic recompilation of heavily used interpreted code*

1170. remote_run/1 : *runs Goal on remote server using default password*

1171. remote_run/3 : *(Host,Port,Goal): runs Goal on remote server at Host, Port with default password*

1172. remote_run/6 : *(Host,Port,Answer,Goal>Password,Reply): runs Goal on server at Host, Port with given Password and returns Reply. However, if you do let(where,here) before calling it, a local goal is called instead.*

1173. remove_current_term/1 : *(Iterator) removes current term attached to an iterator*

1174. repeat/0 : *backtracks until its continuation succeeds; defined as repeat. repeat:-repeat.*

1175. restart/0 : *cleans up data areas and reinitializes symbol tables*

1176. retract/1 : *backtracks over deleting matching clauses*

1177. retract1/1 : *deletes first matching clause in the current database*

1178. retractall/1 : *deletes all matching clauses*

1179. return/1 : *returns data from an engine as if it were an answer - such that the engine can be resumed with get/2 to execute the next goal*

1180. return0/1 : *returns a term from an engine left in a state ready to resume*

1181. reverse/2 : *reverses a list*

1182. **EXAMPLE (S)** :

1183. ?-reverse([a,b,c],_A) .

1184. _A = [c,b,a];

1185.

1186. no

1187. round/2 : *float to int function*

1188. **EXAMPLE (S)** :

1189. ?-round(1.51,_A) .

1190. _A = 2;

1191.

1192. no

1193. rpc/1 : *(Query): calls server on current local reusable socket*

- 1194. **rpc/3** : *(Answer,Goal,Result): calls server on local reusable socket and gets back Result as the(Answer) or no*
- 1195. **rpc_handler/1** : *(Goal): user-defined rpc handler - filters/calls Goal received on server*
- 1196. **rpc_server/0** : *runs Jinni compatible server with socket reuse on default port*
- 1197. **rpc_server/2** : *Port,Password: runs Jinni compatible server with socket reuse*
- 1198. **rpc_server/3** : *Port,Password,Timeout: runs Jinni compatible server with socket reuse*
- 1199. **rpc_test/0** : *tests rpc server and client with socket reuse*
- 1200. **rpc_test/1** : *tests rpc client with socket reuse for a given number of operations*
- 1201. **rtime/1** : *gets elapsed real time in secs*
- 1202. **EXAMPLE (S)** :
- 1203. ? - `rtime(_A).`
- 1204. `_A = 0;`
- 1205.
- 1206. no
- 1207. **run_server/0** : *runs foreground server on localhost (default port) for Jinni clients*
- 1208. **run_server/1** : *runs foreground server on Port to provide services to Jinni clients*
- 1209. **run_server/2** : *(Port,Password): runs server on Port, Password required from clients*
- 1210. **run_server/6** : *(Port,Password,Heap,Stack,Trail,Timeout): runs server with specified service parameters*
- 1211. **scompile/1** : *smart compile/1 variant: if the *.wam file is newer reloads, otherwise fcompiles first*
- 1212. **sconsult/1** : *reconsult/1 variant: cleans up data areas consults, makes all static*
- 1213. **sdebug/1** : *emits mt-safe synchronized debug message*
- 1214. **see/1** : *focuses input on a file*
- 1215. **see_at/1** : *seeks a seekable file at a give offset (in bytes)*
- 1216. **see_or_fail/1** : *opens a file if it exists, otherwise fails*
- 1217. **seeing/1** : *gets file name opened and set by see/1*
- 1218. **seeing_at/1** : *retrieves position in current file opened by see/1*
- 1219. **seen/0** : *close file opened by see/1*
- 1220. **sentence_of/2** : *reads a natural language file to a sentence built as a list of words - on backtracking*
- 1221. **sentence_of/3** : *(File,Ends,Sent) reads a file to a sentence separated by Ends - on backtracking*
- 1222. **service_loop/2** : *ServiceSocket,Password: starts service loop on reusable ServiceSocket - works on server side with server/2*
- 1223. **set_bp_error/4** : *(Id,Mes,Arg1,Arg2): notifies emulator about error condition*

1224. **set_c_threshold/1** : *related to C generator: sets length K of WAM instruction block such that block larger than K will get compiled to C code*

1225. **set_c_threshold/2** : (Min,Max): *related to C generator: sets Min,Max length of WAM instruction block such that blocks between Min and Max size will get compiled to C code*

1226. **set_db/1** : *sets the name of active database for dynamic code*

1227. **set_default/1** : *asserts arg 1 as default state for use by default/2*

1228. **set_hash_max/1** : *Set the range of values from -1 to -Max, that should be a prime number, defaults to 1999*

1229. **set_host/1** : *asserts IP address or name of server host we want to talk to*

1230. **set_input/1** : *sets current input stream*

1231. **set_lineno/1** : *sets line number counter in current file - use with care*

1232. **set_load_method/1** : *sets the current load method by name*

1233. **set_login/1** : *asserts default (nick)name for user*

1234. **set_output/1** : *sets current output stream*

1235. **set_password/1** : *sets default password for user*

1236. **set_port/1** : *asserts port number of the server we want to talk to*

1237. **set_prolog_flag/2** : (Flag,Value): *sets the value of a Prolog flag*

1238. **set_this_host/1** : *asserts IP address or name of this machine*

1239. **set_this_port/1** : *asserts default port for to work as a server on*

1240. **set_timeout/1** : *asserts default timeout*

1241. **setarg/3** : *backtrackable: setarg(I,T,X) replaces arg I of T with X*

1242. **EXAMPLE (S)** :

1243. `?-setarg(2,f(a,b),c).`

1244.

1245. `yes`

1246. **setof/3** : *all solutions predicate generating sorted sets of unduplicated answers*

1247. **EXAMPLE (S)** :

1248. `?-setof(_A,member(_A,[3,2,2,1]),_B).`

1249. `_A = _x61091;`

1250. `_B = [1,2,3];`

1251.

1252. `no`

1253. **shell/1** : *passes a command to the OS*

1254. **show_default/1** : *makes available a default value*

1255. **show_defaults/0** : *show default values of some system variables*

1256. **show_engine/0** : *if debugmes/1 is on (as with quiet(1)), shows params of current engine*

1257. **show_info/2** : *generates components of info/1 output*

1258. **sign/2** : *int function*

1259. **sin/2** : *float function*

1260. **skip_until/2** : *findall variant*

1261. **skip_when/2** : *findall variant*

1262. **sleep/1** : *waits arg 1 seconds*

1263. **sock2file/2** : *reads from a socket, to a file*

1264. **sock_read/2** : *reads from a socket when size of the data is described by int before chars to be read*

1265. **sock_read/3** : *reads from a socket, a string*
 1266. **sock_readln/2** : *reads from a socket until an end of line LF (ascii 10) or char 0 is found and discards possible previous CR (ascii 13)*
 1267. **sock_readln/3** : *reads from a socket, a line*
 1268. **sock_write/2** : *writes a string to a socket prefixed by its lenght*
 1269. **sock_write/3** : *writes to a socket, a string*
 1270. **sock_writeln/2** : *writes a string to a socket and adds an ascii 10 to the end*
 1271. **sock_writeln/3** : *writes to a socket, a line*
 1272. **solutions/2** : *(GX,Xs): adds (last) output arg X to closure G then works like findall(X,GX,Xs)*
 1273. **EXAMPLE (S)** :
 1274. `?-solutions( argn( _A, f( a, b, c) ) , _B) .`
 1275. `_A = _x61092;`
 1276. `_B = [a,b,c];`
 1277. `no`
 1278. `no`
 1279. **sort/2** : *sorts and removes duplicates*
 1280. **EXAMPLE (S)** :
 1281. `?-sort( [2,1,3,1,4,4,2] , _A) .`
 1282. `_A = [1,2,3,4];`
 1283. `no`
 1284. `no`
 1285. **spawn/1** : *spawns Goal in a new bp window on W95/NT PC and Unix/X*
 1286. **spawn/3** : *spawns(Goal,Includes,TempFile): spawns a new bp window*
 1287. **(spy)/1** : *set spy point on goal, triggering trace when interpreted*
 1288. **spying/1** : *checks what we are spying*
 1289. **sqrt/2** : *returns square root of arg 1, a float*
 1290. **EXAMPLE (S)** :
 1291. `?-sqrt( 2, _A) .`
 1292. `_A = 1.41421356;`
 1293. `no`
 1294. `no`
 1295. **sread/2** : *reads a term from a string (atom)*
 1296. **sread/3** : *reads a term and a list of vars from a string (atom)*
 1297. **EXAMPLE (S)** :
 1298. `?-sread( f( X, Y) , _A, _B) .`
 1299. `_A = f( _x62165, _x62193) ;`
 1300. `_B = [X = _x62165, Y = _x62193];`
 1301. `no`
 1302. `no`
 1303. **star/4** : *star(Recognizer,Result): DCG based star regexp processor for (Recognizer)**
 1304. **start_rpc/0** : *starts rpc client on default local reusable socket and port*
 1305. **start_rpc/3** : *(Host,Port,Password): starts rpc client on local reusable socket*
 1306. **stat/0** : *short hand for statistics*
 1307. **stat2dyn/1** : *stat2dyn(H): reverts to uncompiled representation for a predicate with head H*

1308. **stat2dyn/2** : *stat2dyn(Db,H): reverts to uncompiled representation for a predicate in database Db with head H*

1309. **static_prolog_flag/2** : *contains read-only Prolog flag values*

1310. **statistics/0** : *shows info about data areas*

1311. **EXAMPLE (S)** :

```
1312. ?-statistics.  
1313. runtime=[421,156]  
1314. global_stack=[245064,1819260]  
1315. local_stack=[244,260864]  
1316. trail=[33684,227432]  
1317. code=[260404,788156]  
1318. strings=[246,1048330]  
1319. symbols=[13028,249116]  
1320. htable=[113988,672444]  
1321. bboard=[24988,497248]  
1322. gctime=[0,0]  
1323. realtime=[0,343416141]
```

1324.

1325. yes

1326. **statistics/2** : *returns info about data areas*

1327. **EXAMPLE (S)** :

```
1328. ?-statistics(_A,_B).  
1329. _A = runtime;  
1330. _B = [421,0];  
1331.  
1332. _A = global_stack;  
1333. _B = [245600,1818724];  
1334.  
1335. _A = local_stack;  
1336. _B = [228,260880];  
1337.  
1338. _A = trail;  
1339. _B = [33704,227412];  
1340.  
1341. _A = code;  
1342. _B = [260404,788156];  
1343.  
1344. _A = strings;  
1345. _B = [246,1048330];  
1346.  
1347. _A = symbols;  
1348. _B = [13028,249116];  
1349.  
1350. _A = htable;  
1351. _B = [113988,672444];  
1352.  
1353. _A = bboard;  
1354. _B = [24988,497248];  
1355.  
1356. _A = gctime;  
1357. _B = [0,0];  
1358.  
1359. _A = realtime;  
1360. _B = [0,343416141];  
1361.  
1362. no
```

1363. **stop/0** : *exits thread or process*
 1364. **stop/1** : *stops and frees resources held by an engine (may happen automatically if an engine fails)*
 1365. **stop_rpc/0** : *stops rpc client on local reusable socket*
 1366. **stop_service/0** : *stops server with socket reuse - acts on the server side*
 1367. **subsumes_chk/2** : *checks if arg 1 is subsumed by arg 2, after renaming vars*
 1368. **sum/2** : *(List,?Result): sum of a list*
 1369. **EXAMPLE (S)** :
 1370. ?-sum([10,20],_A).
 1371. _A = 30;
 1372. no
 1373. no
 1374. **swap_cont/2** : *calls arg 1 as cc and returns cc in arg 2*
 1375. **swrite/2** : *writes a term to a string (atom)*
 1376. **swrite/3** : *writes a term with a liste of vars to a string (atom)*
 1377. **symcat/3** : *makes new identifier from arg 1 and arg 2*
 1378. **EXAMPLE (S)** :
 1379. ?-symcat(a,b,_A).
 1380. _A = a_b;
 1381. no
 1382. no
 1383. ?-symcat(a,1,_A).
 1384. _A = a_1;
 1385. no
 1386. no
 1387. **synchronize/1** : *wraps Goal in arg 1 for serialized execution*
 1388. **synchronize/2** : *wraps Goal for serialized execution and returns true or fail*
 1389. **synchronize_on/2** : *wraps Goal for sync on given mutex for serialized execution*
 1390. **synchronize_on/3** : *wraps Goal for sync on given mutex for serialized execution and returns true or fail*
 1391. **system/1** : *passes a command to the OS*
 1392. **system/2** : *passes a command to the OS and gets back return code*
 1393. **tab/1** : *outputs N blanks*
 1394. **take_at_most/2** : *(N,Goal) computes at most N answers of Goal*
 1395. **tan/2** : *float function*
 1396. **tell/1** : *focuses output on a file*
 1397. **tell_at/1** : *moves output file pointer to a given offset (in bytes)*
 1398. **tell_at_end/1** : *focuses output on file opened in append mode*
 1399. **telling/1** : *gets file name opened and set by tell/1*
 1400. **telling_at/1** : *retrieves output file position (in bytes)*
 1401. **term_append/3** : *efficiently concatenates 2 terms*
 1402. **EXAMPLE (S)** :
 1403. ?-term_append(f(a,b),g(c,d),_A).
 1404. _A = f(a,b,c,d);
 1405. no
 1406. no

1407. **term_chars/2** : *(Term,Chars): converts between a term and its list of char code representation*

```
1408.      EXAMPLE (S) :  
1409.      ?-term_chars(f(a,b),_A).  
1410.      _A = [102,40,97,44,98,41];  
1411.  
1412.      no  
1413.      ?-term_chars(_A,[102,40,97,44,98,41]).  
1414.      _A = f(a,b);  
1415.  
1416.      no
```

1417. **term_codes/2** : *(Term,Chars): converts between a term and its list of char code representation*

```
1418.      EXAMPLE (S) :  
1419.      ?-term_codes(f(a,b),_A).  
1420.      _A = [102,40,97,44,98,41];  
1421.  
1422.      no  
1423.      ?-term_codes(_A,[102,40,97,44,98,41]).  
1424.      _A = f(a,b);  
1425.  
1426.      no
```

1427. **term_decoder/2** : *(Encrypted,Plain): user provided encoder for secure communications - works on lists of ascii codes*

1428. **term_encoder/2** : *(Plain,Encrypted): user provided decoder for secure communications - works on list of ascii codes*

1429. **term_expansion/2** : *can be used to define a hook into the default DCG expansion mechanism*

1430. **term_hash/2** : *computes hash code on terms ground up to depth 64; fails if something is unbound or the limit is reached*

```
1431.      EXAMPLE (S) :  
1432.      ?-term_hash(t(a,b),_A).  
1433.      _A = 236945626;  
1434.  
1435.      no  
1436.      ?-term_hash(t(a,c),_A).  
1437.      _A = 236945625;  
1438.  
1439.      no
```

1440. **term_of/2** : *reads a Prolog file to a set of terms - on backtracking*

1441. **term_store_op/4** : *API for external term storage*

1442. **textedit/0** : *calls textedit editor on last compiled file*

1443. **the/1** : *defined as the(G):-the(G,G)*

1444. **the/2** : *defined as the(X,G):-the(X,G,the(X))*

1445. **the/3** : *the(X,G,R) first answer R=the(X) or R=no, without binding G*

1446. **this_db/1** : *same as current_db, returns the current database*

1447. **this_engine/1** : *gets a handle to the current engine*

1448. **this_host/1** : *assumes default IP adress or name this machiner*

1449. **this_port/1** : *assumes default port for to work as a server on*

1450. **thread_cancel/1** : *terminates (cancels) thread given in arg 1*

1451. **thread_exit/0** : *exits a thread*

1452. **thread_exit/1** : *exits a thread*
 1453. **thread_join/1** : *joins thread*
 1454. **thread_notify/1** : *notifies a thread waiting on guard with thread_wait*
 1455. **thread_notify_all/1** : *notifies all threads waiting on guard with thread_wait*
 1456. **thread_resume/1** : *resumes execution of suspended thread*
 1457. **thread_suspend/1** : *suspends execution of thread*
 1458. **thread_timed_wait/2** : *waits on first arg Guard second arg msec or until notified*
 1459. **thread_wait/1** : *waits on guard until notified by thread_notify*
 1460. **throw/1** : *ISO Prolog exception operator: throws a term to be caught by a matching catch*
 1461. **throw_with_cont/2** : *works for throw/1: used to rethrow with new continuation*
 1462. **timed_call/4** : *(Answer,Goal,Timeout,Result) - calls and possibly stops Goal after Timeout secs*
 1463. **timeout/1** : *assumes default timeout*
 1464. **to_engine/2** : *(E,T) sends to engine E a term T, and fails if E has an empty message box*
 1465. **to_lower_char/2** : *(Upper,Lower): converts a char to lower case*
 1466. **to_lower_chars/2** : *converts a list of chars to lower case*
 1467. **to_string/2** : *converts a term to a string constant (in particular, converts numbers to strings)*
 1468. **EXAMPLE (S)** :
 1469. `?-to_string(13,_A).`
 1470. `_A = 13;`
 1471. `no`
 1472. `no`
 1473. `?-to_string(3.14,_A).`
 1474. `_A = 3.14;`
 1475. `no`
 1476. `no`
 1477. `?-to_string(f(_A,_A),_B).`
 1478. `_A = _x61099;`
 1479. `_B = f(_x61099,_x61099);`
 1480. `no`
 1481. `no`
 1482. **to_tokens/2** : *to_tokens(Codes,PrologTokens)*
 1483. **to_upper_char/2** : *(Lower,Upper): converts a char to upper case*
 1484. **to_upper_chars/2** : *converts a list of chars to upper case*
 1485. **to_word_codes/2** : *to_word_codes(Codes,ListOfWordCodes)*
 1486. **to_words/2** : *to_words(Codes,NaturalLanguageWords)*
 1487. **token_of/2** : *reads a Prolog file to a set of tokens - on backtracking*
 1488. **tokens_of/2** : *reads a Prolog file to a slist of tokens*
 1489. **told/0** : *closes file opened by tell/1*
 1490. **topcall/1** : *calls arg 1 as if it were entered by the user at Prolog prompt*
 1491. **toplevel/0** : *interactive toplevel Prolog loop*
 1492. **topstep/1** : *interactive toplevel Prolog step*
 1493. **trace/0** : *trace all predicates when interpreted*

1494. **trace/1** : *traces execution of a goal*
 1495. **trim_term/4** : *trim_term(D,Filler,T,NewT) replaces subterms of T deeper than D with Filler*
 1496. **true/0** : *always succeeds*
 1497. **truncate/2** : *float to int function*
 1498. **EXAMPLE (S)** :
 1499. `?-truncate(1.51,_A).`
 1500. `_A = 1;`
 1501.
 1502. `no`
 1503. **try_unlock_thread_guard/1** : *try to unlock a thread guard*
 1504. **try_unlock_thread_guard/2** : *try to unlock a thread guard for a specified timeout*
 1505. **tstest/3** : *(K,V,R): performs various tests with external terms*
 1506. **tsync_op/3** : *same as thread_operation(Op, MutexOrParam, ActionOrValue) various thread synchronization operations*
 1507. **ttyprin/1** : *writes to terminal*
 1508. **ttyprint/1** : *writes to terminal with a new line*
 1509. **unix/1** : *executes various Unix commands*
 1510. **unix_access/2** : *checks if arg1 (a path+file) is accessible in arg 2 (integer) mode*
 1511. **unix_argc/1** : *gets cmd line arg counter*
 1512. **EXAMPLE (S)** :
 1513. `?-unix_argc(_A).`
 1514. `_A = 2;`
 1515.
 1516. `no`
 1517. **unix_argv/1** : *gets the list of cmd line args from 1 to argc*
 1518. **EXAMPLE (S)** :
 1519. `?-unix_argv(_A).`
 1520. `_A = [helpgen.pro];`
 1521.
 1522. `no`
 1523. **unix_argv/2** : *gets a cmd line arg from 0 to argc*
 1524. **EXAMPLE (S)** :
 1525. `?-unix_argv(0,_A).`
 1526. `_A = bp;`
 1527.
 1528. `no`
 1529. **unix_cat/1** : *prints a file to user terminal*
 1530. **unix_cd/1** : *changes local dir to arg 1*
 1531. **unix_fork/1** : *starts child process with Unix fork*
 1532. **unix_getenv/2** : *gets an environment variable*
 1533. **unix_kill/2** : *sends signal arg 1 to process with pid arg 2*
 1534. **unix_pid/1** : *returns process id of current process*
 1535. **unlock_thread_guard/1** : *unlocks thread guard*
 1536. **untrail_to/1** : *unwinds the trail up to a choice point*
 1537. **update_current_term/2** : *(Iterator,NewTerm): replaces current term of an iterator*
 1538. **user_error/2** : *writes basic error message and fail*

1539. **var/1** : *true if currently an unbound variable*

1540. **variant_of/2** : *checks if args are the same up to a renaming of vars*

1541. **vars_of/2** : *(Term,Vars): lists free vars of a term*

1542. **EXAMPLE (S)** :

1543. `?-vars_of(f(_A,t(_A,_B,_C),_C),_D).`

1544. `_A = _x61092;`

1545. `_B = _x61097;`

1546. `_C = _x61094;`

1547. `_D = [_x61092,_x61097,_x61094];`

1548.

1549. `no`

1550. **vget/2** : *gets a VSHARE declared C data object in a term like int(N),F/N or var(V)*

1551. **vget0/2** : *gets the type and value of a C word*

1552. **vget_int/2** : *gets a VSHARE defined int C variable to a 28 bit int*

1553. **vget_int0/2** : *gets from arg 1 - an int* C variable - to a 28 bit int*

1554. **vi/0** : *calls vi editor on last compiled file*

1555. **vread/2** : *reads HDEFI or HDEFS defined C constant into a Prolog integer or atom*

1556. **vset/2** : *sets a VSHARE declared C data object in a term like int(N),F/N or var(V)*

1557. **vset_int/2** : *sets a VSHARE defined int C variable to a 28 bit int*

1558. **vset_int0/2** : *sets arg 1 - an int* C variable - to a 28 bit int*

1559. **wait_for/2** : *wait_for(Term,Constraint) waits for a term on the blackboard, such that Constraint holds*

1560. **while/2** : *(Cond,Goal): findall variant which explores alternative answers for Goal, while Cond holds,*

1561. **words_code/2** : *generates a code ready to print, one a t a time with put/1 from a list of words*

1562. **write/1** : *writes to current output stream set with tell/1, defaults to - Prolog's stdio*

1563. **write_chars/1** : *writes a list of ascii codes*

1564. **EXAMPLE (S)** :

1565. `?-write_chars([104,101,108,108,111]).`

1566. `hello`

1567. `yes`

1568. **write_term_to_chars/2** : *writes a term to a list of char codes*

1569. **EXAMPLE (S)** :

1570. `?-write_term_to_chars(f(_A,_A,_B,_B),_C).`

1571. `_A = _x61095;`

1572. `_B = _x61097;`

1573. `_C =`
`[102,40,95,120,54,49,48,57,53,44,95,120,54,49,48,57,53,44,95,120,`
`54,49,48,57,55,44,95,120,54,49,48,57,55,41];`

1574.

1575. `no`

1576. **write_term_to_chars/3** : *writes a term with variables names to a list of char codes*

1577. **EXAMPLE (S)** :

1578. `?-write_term_to_chars(f(_A,_B),[X=_A,Y=_B],_C).`

1579. `_A = _x61098;`

```

1580.      _B = _x61099;
1581.      _C = [102, 40, 39, 88, 39, 44, 39, 89, 39, 41];
1582.
1583.      no
1584.      write_words/1 : write list of words to a space separated line
1585.      writeln/1 : variant of write which quotes if needed, so that term is read
                     back correctly/1
1586.      x_abolish/1 : db hook, deletes all matching clauses and unmarks the
                     clause as dynamic
1587.      x_asserta/1 : db hook, adds a clause to be first in a predicate definition
1588.      x_assertz/1 : db hook, adds a clause to be last in a predicate definition
1589.      x_clause/2 : db hook, clause(H,B) generates a clause with head matching
                     H and body B
1590.      x_consult/1 : db hook, consults with possible duplication of clauses
1591.      x_dynamic/1 : db hook, states that a predicate can be updated
1592.      x_gc/0 : db hook, recovers memory used by the database
1593.      x_is_dynamic/1 : db hook, checks if dynamic
1594.      x_listing/0 : db hook, lists given predicate if in current database
1595.      x_listing/1 : lists predicate F of arity N if in current database
1596.      x_profile/0 : db hook, shows the result of profiling compiled predicates
1597.      x_retract/1 : db hook, backtracks over deleting matching clauses
1598.      x_retractall/1 : db hook, deletes all matching clauses
1599.      x_trace/1 : traces and profiles predicates, in combination with x_profile
1600.      x_trace/2 : db hook, internal
1601.      x_traced_call/1 : traces calls if compile-time x_trace triggers it
1602.      xload/1 : xload(File) loads clauses to the current database, from a binary
                     File
1603.      xsave/1 : xsave(File) saves all the clauses of the current database to a
                     binary File
1604.      ~ /1 : short hand for reconsult

```